

Видео как сигнал: от пикселей к битам

Видео — дискретная последовательность изображений, представляющая моментальные срезы сцены. Цифровое видео — это дискретизация непрерывного оптического сигнала по пространству и времени.

$$I[m, n, k] = I(x_m, y_n, t_k)$$

где Δx , Δy — шаги дискретизации по пространству, $\Delta t = 1/f$ — временной шаг. Дискретизация влечёт ошибку квантования и ограничение по частоте Найквиста.

Частота кадров и экспозиция

Временная интерференция

Каждый кадр формируется через оптическое интегрирование в течение времени затвора (shutter time). Долгая экспозиция создаёт motion blur, короткая — рывки при низком FPS.

$$I[m, n, k] = \frac{1}{T_s} \int_{t_k - T_s/2}^{t_k + T_s/2} I(x_m, y_n, t) dt$$

Баланс параметров

Видео с частотой 30 fps и затвором 1/60 с имеет перекрытие во времени — визуальная плавность. Баланс между FPS и экспозицией определяет SNR и динамический диапазон камеры.

Алиасинг: искажения дискретизации

Пространственный алиасинг

При низком разрешении возникает муар и ложные детали

$$f_x < \frac{1}{2\Delta x}, \quad f_y < \frac{1}{2\Delta y}$$

Временной алиасинг

Недостаточный FPS создаёт эффект «колёс назад» — движение кажется реверсным при превышении $f/2$

Решение

Антиалиасинговые фильтры (оптические low-pass) и электронное сглаживание сигнала до оцифровки

CFR и VFR: частота кадров

Два режима

- **CFR (Constant Frame Rate)** — кадры через равные интервалы $\Delta t = 1/f$
- **VFR (Variable Frame Rate)** — интервалы различаются, каждый кадр имеет PTS

Параметр `timebase` задаёт единицу времени внутри контейнера (например, $1/90000$ для MPEG-TS).

$$t = \frac{PTS}{timebase}$$

📌 **Критично для ML:** VFR может ломать обучение моделей из-за неравномерного stride между кадрами

PTS, DTS и временные метки

1

PTS

Presentation Timestamp — когда кадр должен быть отображён

2

DTS

Decoding Timestamp — когда кадр должен быть декодирован.
Для В-кадров: DTS < PTS

3

Latency

Задержка между захватом и отображением

$$L = t_{display} - t_{capture}$$

4

Jitter

Вариация задержки

$$J = \sqrt{\frac{1}{N} \sum_i (L_i - \bar{L})^2}$$

GOP-структура

Group of Pictures

Последовательность от I-кадра до следующего I-кадра. P- и B-кадры ссылаются на другие кадры, создавая задержку декодирования.

При $GOP = 30$ и $FPS = 30$ задержка составляет ~1 секунду между захватом и отображением.

Низколатентные приложения

Для видеоаналитики в реальном времени GOP сокращают или используют режим **intra-only** (H.264 All-I).

Влияние параметров съёмки

VFR и обучение

Переменная частота кадров ломает модели action recognition — stride между кадрами становится неоднородным

Motion blur

Снижает эффективность детекторов границ, но улучшает обучение моделей оптического потока

Камера как элемент системы

Параметры съёмки влияют на качество данных для нейросетей — камера не просто датчик

Метрики качества видео



PSNR

Peak Signal-to-Noise Ratio

$$PSNR = 10 \log_{10} \frac{MAX_I^2}{MSE}$$

Высокий PSNR (> 40 dB) — почти идеальное воспроизведение



SSIM

Structural Similarity Index

$$SSIM = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

Оценивает структурное сходство



VMAF

Video Multimethod Assessment Fusion (Netflix)

$$VMAF = f(PSNR, SSIM, VIF, DLM)$$

Лучше коррелирует с человеческим восприятием

Время как измерение

Для компьютерного зрения видео добавляет ещё одно измерение — время. Темпоральные связи становятся равноправными пространственным.



Модель, способная выделять регулярности по оси t , понимает действие, а не просто форму.

Видео — это не просто много картинок, а сигнал, где время несёт информацию. Эта интуиция важна для всей современной видеоаналитики.

От RGB к Y'CbCr

Идея декорреляции

RGB-компоненты сильно коррелированы: изменение яркости влияет на все три канала. Переход к Y'CbCr разделяет яркость и цвет.

- **Y'** — яркость (luma)
- **Cb, Cr** — отклонения от серого в синем и красном направлениях

Преобразование BT.709

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.2126 & 0.7152 & 0.0722 \\ -0.1146 & -0.3854 & 0.5000 \\ 0.5000 & -0.4542 & -0.0458 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Коэффициенты отражают чувствительность зрения к цветам

Обратное преобразование Y'CbCr → RGB

Восстановление RGB-компонент выполняется по обратному преобразованию:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.000 & 0.000 & 1.5748 \\ 1.000 & -0.1873 & -0.4681 \\ 1.000 & 1.8556 & 0.000 \end{bmatrix} \begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix}$$

❏ **Важно:** При несовпадении матрицы кодирования и декодирования возникают ошибки цветопередачи — зелёные участки приобретают пурпурный оттенок

Стандарты цветковых пространств



Несоответствие стандартов приводит к систематическим сдвигам в распределении пикселей. На этапе препроцессинга важно нормализовать видео в единый цветовой стандарт.

Хромасабсэмплинг

4:4:4

Полное разрешение

Все компоненты хранятся с одинаковым разрешением

4:2:2

Половинное по горизонтали

Цветовые каналы имеют половинное горизонтальное разрешение

4:2:0

Половинное по обеим осям

На 4 пикселя — одна пара Cb/Cr.
Уменьшает объём данных вдвое

Человеческий глаз чувствителен к яркости больше, чем к цвету. Для задач сегментации предпочтительно использовать 4:4:4.

Гамма-коррекция

Перцептивное кодирование

Сенсор измеряет свет линейно, но восприятие нелинейно. Гамма-коррекция перераспределяет уровни яркости:

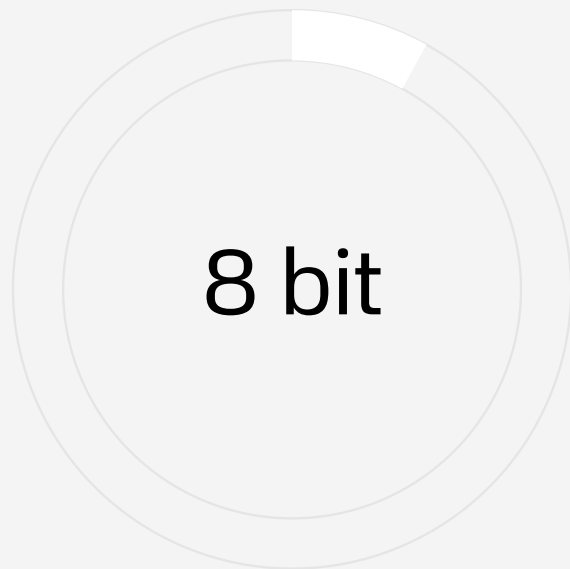
$$V_{out} = V_{in}^{1/\gamma}$$

где $\gamma \approx 2.2$ для большинства дисплеев



Для нейросетей: Большинство моделей обучены на линейных RGB. Данные из sRGB или BT.709 нужно «разгаммировать» — возвести пиксели в степень γ

Битовая глубина и квантование



256 уровней яркости. Стандарт для большинства датасетов CV



1024 уровня. Используется в HDR, меньше banding-эффектов



4096 уровней. Максимальная точность для профессионального видео

Количество уровней: 2^b , где b — битовая глубина. Высокая битность позволяет воспроизводить детали в тенях и светах.

Конвертация YUV → RGB через PyAV

```
import av
import numpy as np

container = av.open("video.mp4")
for frame in container.decode(video=0):
    rgb = frame.to_rgb().to_ndarray()
    yuv = frame.to_ndarray(format="yuv420p")
    # rgb: H×W×3, yuv: packed planar array
```

Метод `to_rgb()` выполняет декодирование с учётом матрицы преобразования (BT.709/601) и хромасабсэмплинга. Обеспечивает корректное представление цветов.

Цветовые артефакты

Неправильная декодировка цветового пространства способна радикально исказить распределение данных

Перепутанные матрицы BT.601 ↔ BT.709 создают зелёный оттенок, который модели интерпретируют как новый класс текстуры.

15-20%

Снижение точности

При неправильной декодировке в action recognition

В продакшене декодирование выполняется строго по метаданным контейнера с явным указанием `-color primaries` и `-color trc` в FFmpeg.

HDR и расширенные цветовые пространства

Современные стандарты

HDR10, Dolby Vision используют расширенные цветовые охваты BT.2020 и PQ (Perceptual Quantizer) кривые.

Для нейросетевых систем с обычными мониторами такие видео необходимо конвертировать в sRGB.

Обработка HDR

Ошибочная обработка HDR-данных (прямое чтение PQ-кодированных значений) приводит к потере контраста и инверсии цветов.

Требуется полный конверт в линейное пространство или обучение модели на том же цветовом профиле.

Контейнер и кодек

Контейнер

Формат хранения и мультиплексирования потоков: видео, аудио, субтитров, метаданных

Примеры: MP4, MKV, MOV, AVI, MPEG-TS

Кодек

Алгоритм кодирования и декодирования видеопоследовательности

Примеры: H.264/AVC, H.265/HEVC, VP9, AV1, H.266/VVC

Контейнер — «упаковка», кодек — «язык». Один контейнер может содержать потоки разных кодеков. Проверяйте fourcc-код в метаданных!

Внутрикадровое и межкадровое сжатие

1

Intra-frame

Устранение пространственной избыточности внутри кадра.
Аналогично JPEG: DCT + квантование + энтропийное кодирование

2

Inter-frame

Устранение временной избыточности между кадрами.
Предсказание движения: новый кадр = отклонение от предыдущего + вектор движения

Межкадровое сжатие снижает объём данных в десятки раз, но делает поток зависимым от предшествующих кадров.

Типы кадров: I, P, B



I-кадры (Intra)

Независимые, полное изображение.
Точки входа в поток



P-кадры (Predicted)

Кодируются как разница с предыдущим
I/P кадром



B-кадры (Bi-predicted)

Интерполяция между предыдущим и
последующим кадром. Высокая
эффективность, но задержка

Порядок отображения $\neq$ порядок декодирования. Требуется буферизация (DPB). Для реального времени часто выбирают GOP без B-кадров.

GOP-структура

Group of Pictures

Последовательность от одного I-кадра до следующего. Пример:

IBBPBBPBBI

Чем длиннее GOP, тем выше степень межкадрового сжатия и больше задержка при декодировании.

Компромисс

При GOP=60 кодек ссылается на кадры, отстоящие на 2 секунды — неприемлемо для систем с ограничением latency.

В видеоаналитике: короткий GOP (8–12) или All-Intra

Rate–Distortion компромисс

Оптимизация кодека — минимизация функции:

$$J = D + \lambda R$$

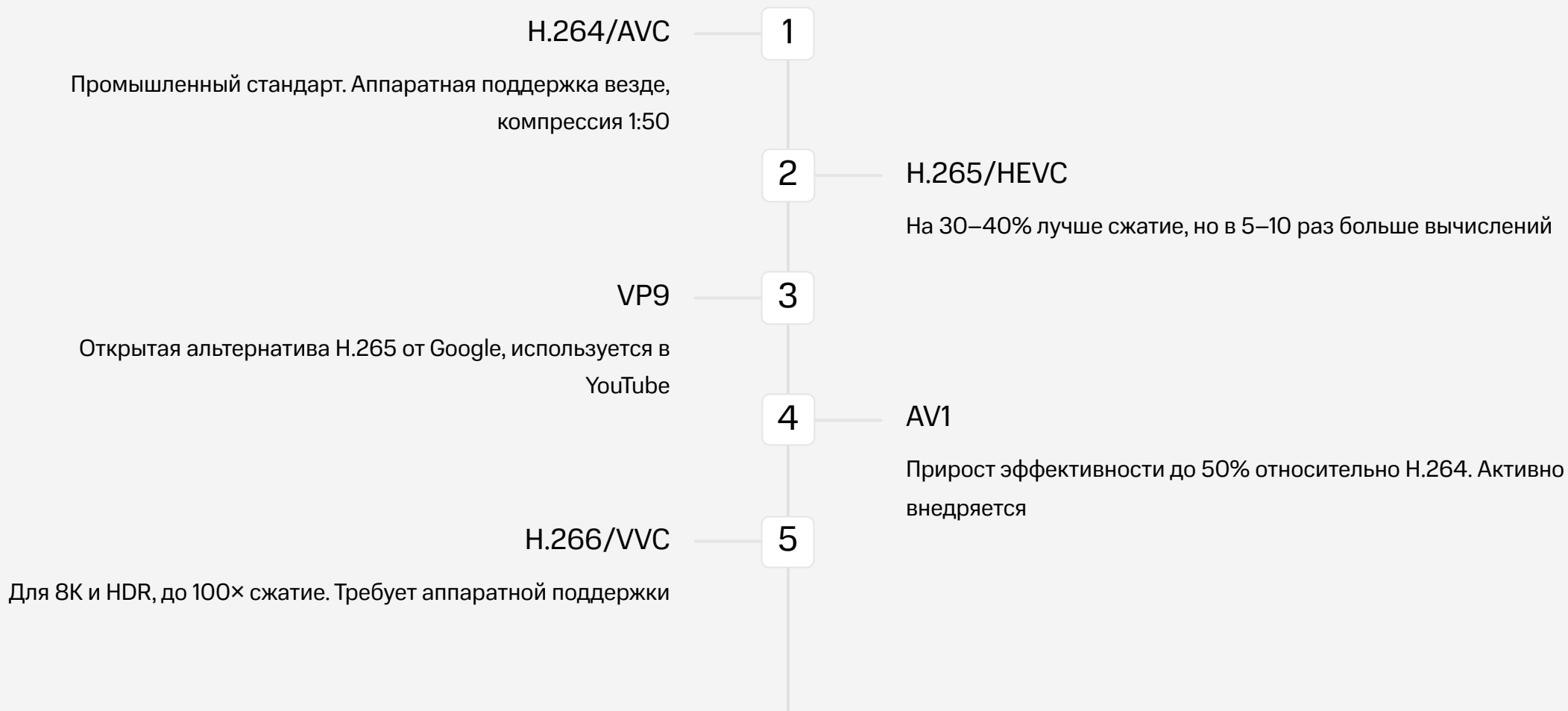
где R — средний битрейт (бит/с), D — среднее искажение (MSE), λ — параметр компромисса

Кривая зависимости битрейта от искажений:

$$R(D) \approx \alpha \log \frac{1}{D}$$

В реальных системах контролируется через **CRF** (Constant Rate Factor) или **QP** (Quantization Parameter): уменьшение QP повышает качество, но увеличивает битрейт экспоненциально.

Основные кодеки



Параметры FFmpeg

Кодирование H.264

```
ffmpeg -i input.mp4 -c:v libx264 \  
-preset slow -crf 22 -g 12 \  
-pix_fmt yuv420p -profile:v high \  
output.mp4
```

- `-crf` — компромисс качество/битрейт (меньше = лучше)
- `-g` — длина GOP
- `-preset` — скорость кодирования
- `-profile` и `-level` — совместимость с аппаратным декодером

Кодирование AV1

```
ffmpeg -i input.mp4 -c:v libaom-av1 \  
-crf 30 -b:v 0 -row-mt 1 output.mkv
```

Артефакты компрессии

Типы искажений

- Блочные границы
- Ringing (звон вокруг контуров)
- Размытые детали
- Потеря текстур

Влияние на CV-модели

Для свёрточных сетей артефакты искажают распределение признаков. Блочные паттерны интерпретируются как структура, а не шум.

При обучении рекомендуется использовать одинаковые параметры сжатия для train/test.

Оценка качества компрессии

Вычисление PSNR через FFmpeg

```
ffmpeg -i ref.mp4 -i test.mp4 \  
-lavfi psnr="stats_file=psnr.log" \  
-f null -
```

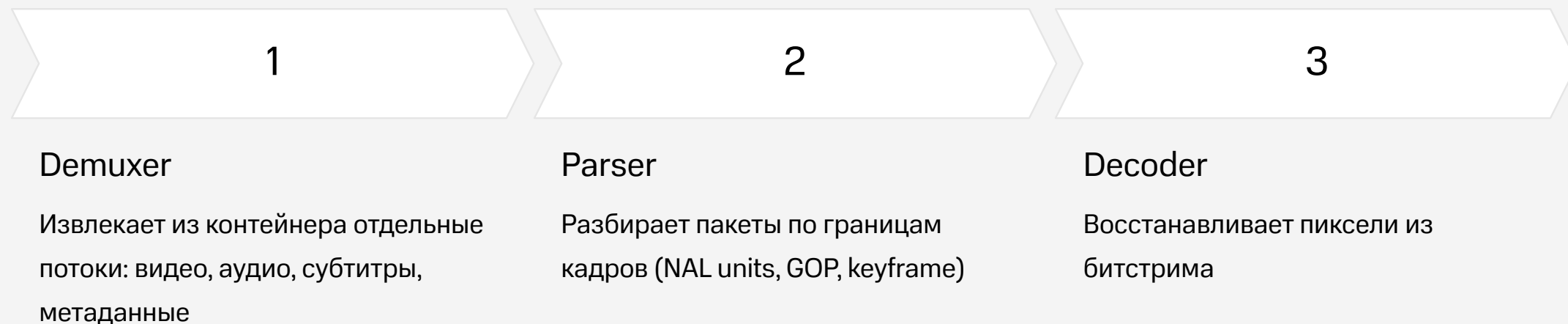
Вычисление PSNR в Python

```
import cv2, numpy as np  
  
def psnr(a, b):  
    mse = np.mean((a.astype(np.float32) -  
                    b.astype(np.float32)) ** 2)  
    return 10 * np.log10((255 ** 2) / mse)
```

Вычисление VMAF

```
ffmpeg -i ref.mp4 -i test.mp4 \  
-lavfi libvmaf="model_path=vmaf_v0.6.1.json" \  
-f null -
```

Архитектура демультимплексора



Демультимплексор работает асинхронно: данные приходят неравномерно, требуется буферизация. При чтении RTSP-потока пакеты видео и аудио поступают с разными задержками.

Буфер кадров (DPB)

Decoded Picture Buffer

Внутренний буфер декодера, содержащий не только последний кадр, но и опорные, нужные для декодирования P- и B-кадров.

Размер зависит от GOP-структуры: для H.264 High Profile — до 16 опорных кадров.

Две функции

1. **Reordering buffer** — хранит кадры, ожидающие отображения
2. **Reference buffer** — обеспечивает доступ к предыдущим для компенсации движения

Важно очищать DPB при seek-операциях

Аппаратное декодирование

Программное (CPU)

Универсальность, но высокое потребление ресурсов

Аппаратное (GPU/ASIC)

NVDEC (NVIDIA), VA-API (Intel), VideoToolbox (Apple)

Высокая пропускная способность при низкой нагрузке CPU

Пример вызова аппаратного декодера

```
ffmpeg -hwaccel cuda \  
-hwaccel_output_format cuda \  
-i input.mp4 -f rawvideo \  
-pix_fmt rgb24 pipe:1
```

Кадры остаются в памяти GPU. Zero-copy снижает latency на 30–50%.

Реордеринг кадров

Из-за В-кадров порядок декодирования (DTS) $\neq$ порядок отображения (PTS)

Последовательность вывода I_0, B_1, B_2, P_3 декодируется как I_0, P_3, B_1, B_2

Декодер должен ждать будущие кадры, прежде чем вывести текущий. В PyAV или OpenCV порядок автоматически выравнивается по PTS.

❏ При прямом доступе к буферу (`avcodec_receive_frame`) инженер должен сам следить за временными метками

Сэмплинг равномерных клипов при VFR

При переменной частоте кадров для получения T равномерно распределённых кадров интерполируем по временным меткам:

$$t_i = i \cdot \frac{T_{vid}}{T - 1}, \quad i = 0, 1, \dots, T - 1$$

```
import av

def sample_uniform(path, num_frames=16):
    container = av.open(path)
    stream = container.streams.video[0]
    duration = stream.duration * stream.time_base
    timestamps = [i * duration / (num_frames - 1)
                  for i in range(num_frames)]
    frames = []
    for frame in container.decode(video=0):
        t = frame.pts * stream.time_base
        if timestamps and t >= timestamps[0]:
            frames.append(frame.to_ndarray(format="rgb24"))
            timestamps.pop(0)
    return frames
```


Zero-copy pipeline

Проблема

Видеопотоки обрабатываются на GPU, но декодер возвращает кадры в память CPU. Копирование между ними — узкое место.

Решение

Декодер пишет кадры сразу в GPU-буфер (pinned memory), который передаётся в модель без промежуточного копирования.

```
ffmpeg -hwaccel cuda -i input.mp4 \  
-vf scale_cuda=640:360,hwdownload,\  
format=rgb24 output.raw
```

Важно следить за выравниванием памяти (stride, pitch) и форматом (NV12 → RGB).

Извлечение кадров по PTS

Пример извлечения кадров с шагом 1 секунда по реальному времени видео:

```
import av

container = av.open("input.mp4")
stream = container.streams.video[0]
time_base = stream.time_base
target_pts = [i / time_base.denominator
               for i in range(0, int(stream.duration *
                                     time_base.denominator),
                               int(1 / time_base))]
frames = []
for packet in container.demux(stream):
    for frame in packet.decode():
        t = frame.pts * time_base
        if target_pts and abs(t - target_pts[0]) < 0.05:
            frames.append(frame.to_ndarray(format="rgb24"))
            target_pts.pop(0)
```

Код учитывает реальный time_base и корректно обрабатывает VFR.

Профилировка пайплайна

Измерение времени

```
import time

t0 = time.time()
frames = sample_uniform("video.mp4", 32)
t1 = time.time()
print(f"Decoding time: {t1 - t0:.3f}s")
print(f"FPS: {len(frames)/(t1-t0):.2f}")
```

Продвинутое профилирование

Используйте `torch.cuda.Event` или NVIDIA Nsight Systems для измерения overlap CPU ↔ GPU и загрузки видеodeкодера (NVDEC utilization).

Цель: полное перекрытие декодирования и инференса

Синхронизация аудио и видео

В потоках RTSP и контейнерах MP4 синхронизация A/V влияет на стабильность таймингов. Если аудио задерживается, видеоплеер может «дропать» кадры.

📌 При подготовке данных для моделей звук лучше удалять (`-an`) или фиксировать таймбазу через `-vsync 0`

Несогласованность A/V может вызвать дрейф timestamp и привести к неоднородному stride между кадрами.

Формат тензора для нейросетей

Стандартное представление видеоклипов — четырёхмерный тензор размера **$T \times C \times H \times W$**

- **T** — количество последовательных кадров
- **C** — число каналов (обычно 3 для RGB)
- **H, W** — пространственные размеры

Преобразование к PyTorch-формату

```
clip = np.stack(frames, axis=0) # T x H x W x C
clip = torch.from_numpy(clip).permute(0,3,1,2)
# T x C x H x W
```

На вход модели подаётся тензор формы **(B, T, C, H, W)** , где **B** — размер батча.

Temporal jitter

Проблема

Простое равномерное сэмплирование делает модель чувствительной к фиксированному FPS.

Решение

Случайное смещение начала клипа и случайный stride между кадрами:

$$i_k = i_0 + k \cdot s + \epsilon_k, \quad \epsilon_k \sim U(-1, 1)$$

Разрушает периодичность, делает модель устойчивой к вариациям FPS

При обучении используется jitter, при валидации — фиксированный stride.

Sliding window

Видео имеет произвольную длину, поэтому обработка происходит по клипам — скользящим окнам фиксированной длины T .

Количество клипов при общем числе кадров N :

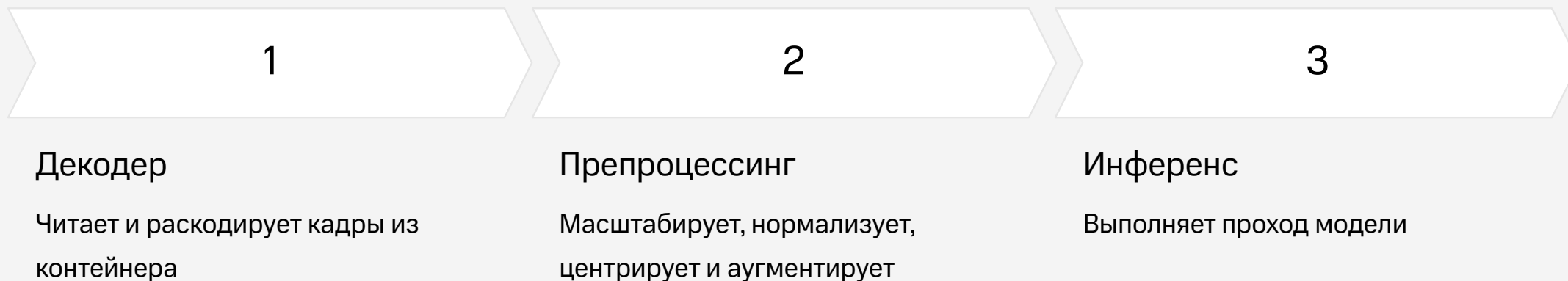
$$K = \left\lfloor \frac{N - T}{S} \right\rfloor + 1$$

где S — шаг окна (stride)

Пример: при $N=300$, $T=32$, $S=16$ получаем $K=17$ клипов

В системах реального времени sliding window применяется для анализа скользящего временного фрагмента потока (например, последние 2 секунды с шагом 0.5 с).

Конвейер обработки видео



Для стабильной производительности стадии должны работать асинхронно, используя очереди. Producer–consumer схема позволяет перекрывать операции по времени и избегать простоев GPU.

DataLoader с параллельным декодированием

```
from torch.utils.data import Dataset, DataLoader
import av, torch, numpy as np

class VideoClipDataset(Dataset):
    def __init__(self, paths, clip_len=16, transform=None):
        self.paths = paths
        self.clip_len = clip_len
        self.transform = transform

    def __getitem__(self, idx):
        path = self.paths[idx]
        container = av.open(path)
        frames = []
        for frame in container.decode(video=0):
            frames.append(frame.to_rgb().to_ndarray())
            if len(frames) == self.clip_len:
                break
        clip = torch.from_numpy(np.stack(frames))
        clip = clip.permute(0,3,1,2).float() / 255.0
        return clip if self.transform is None \
            else self.transform(clip)

    def __len__(self):
        return len(self.paths)

loader = DataLoader(VideoClipDataset(video_files),
                    batch_size=4, num_workers=4,
                    pin_memory=True)
```

Метрики производительности

L

Latency

Время обработки одного клипа

$$L = \frac{1}{N} \sum_{i=1}^N t_i$$

T

Throughput

Количество клипов в секунду

$$T = \frac{N}{\sum_i t_i}$$

J

Jitter

Вариация latency

$$J = \sqrt{\frac{1}{N} \sum_i (t_i - L)^2}$$

```
import time
start = time.time()
for batch in loader:
    _ = model(batch.cuda(non_blocking=True))
elapsed = time.time() - start
latency = elapsed / len(loader)
throughput = len(loader) / elapsed
```

Prefetch и overlap

Концепция

Prefetch — подготовка следующей партии кадров, пока GPU выполняет инференс по предыдущей.

Overlap достигается через многопоточность или CUDA-стримы.

Реализация

```
stream = torch.cuda.Stream()
with torch.cuda.stream(stream):
    next_batch = next(iter(loader))
    next_batch = next_batch.cuda(
        non_blocking=True)
# параллельно GPU выполняет
# инференс предыдущего batch
```

Такой приём снижает эффективную латентность почти вдвое и повышает стабильность FPS.

Надёжность пайплайна

При работе с сотнями видеофайлов нередко ошибки: битые контейнеры, пропущенные кадры, различие таймбаз.

Обработка исключений

Блок try/except при декодировании,
fallback на пропуск ролика

Логирование

Запись ошибок в отдельный файл
для анализа

Воспроизводимость

Фиксация версии FFmpeg и
таймбазы для избежания
несовместимостей

Контроль стабильности FPS

Во время инференса в потоковом режиме важно сохранять стабильный FPS. Если декодер не успевает, система должна адаптироваться.

Frame skipping: при превышении latency текущий кадр отбрасывается, чтобы синхронизация не нарушалась

Это предпочтительнее буферизации для систем реального времени (видеонаблюдение). Адаптивная стратегия может также снижать разрешение при высокой нагрузке.

Практические грабли

Переходим к анализу типичных ошибок и проблем, возникающих в реальных системах видеоаналитики. Понимание этих «граблей» критично для построения надёжных пайплайнов.

Несогласованность цветowych метаданных

Проблема

Два ролика выглядят одинаково, но имеют разные color primaries: один bt709, другой bt2020. FFmpeg применяет дефолтную матрицу — систематическое смещение цветов.

Для нейросетей это дрейф статистики пикселей и ухудшение результатов.

Решение

Явно фиксировать параметры цветового профиля:

```
ffmpeg -i input.mp4 \  
-vf colorspace=all=bt709:\  
iall=bt709:format=yuv420p \  
output.mp4
```

Контролировать chroma_location

Потери кадров из-за GOP-реордеринга

При неправильной обработке потоков можно потерять часть кадров, особенно если обрезка (-ss, -to) производится не по ключевым кадрам.

Точное извлечение участка видео

```
ffmpeg -ss 00:00:02.500 -to 00:00:07.000 \  
-i input.mp4 -c copy \  
-avoid_negative_ts 1 -accurate_seek \  
output.mp4
```

❏ Без `-accurate_seek` можно потерять начальные кадры или получить смещение PTS — несовпадение длины клипа и временных меток

Узкие места PCIe

32 GB/s

Пропускная способность

PCIe 4.0 x16

14 GB/s

Требуется для 1080p@60fps

При float32-данных (3×1920×1080×4 байта)

Даже при идеально оптимизированном декодере производительность может упереться в шину PCIe при копировании между CPU и GPU.

Решение: аппаратное декодирование NVDEC → препроцессинг через CUDA → инференс. Zero-copу исключает возврат данных на CPU

Баланс загрузки GPU vs CPU



Если декодирование полностью аппаратное, CPU простаивает. Если программное — GPU ждёт кадры. Современные пайплайны строятся по принципу *pipeline parallelism*: каждый блок работает независимо, используя очереди с буфером фиксированного размера.

Контроль воспроизводимости

Проблема таймбазы

Видеофайлы не гарантируют стабильную таймбазу: один контейнер может хранить разные значения `time_base` при перекодировке.

Это меняет интерпретацию PTS и влияет на длину извлечённых клипов.

Фиксация таймбазы

```
ffmpeg -i input.mp4 \  
-video_track_timescale 90000 \  
output.mp4
```

Или сохранять `stream.time_base` в метаданных датасета.
При извлечении кадров проверять совпадение с ожидаемой.

Мини-бенчмарки для регрессий

В продакшен-пайплайнах качество видеоаналитики необходимо отслеживать автоматически через короткие скрипты, проверяющие воспроизводимость и стабильность.

Вычисление среднего PSNR

```
ffmpeg -i ref.mp4 -i test.mp4 \  
-lavfi psnr="stats_file=bench.log" \  
-f null -
```

В Python

```
def benchmark_psnr(videos):  
    return [psnr(ref, test)  
            for ref, test in zip(videos[:-1],  
                                videos[1:])]
```

Такие тесты выполняются после обновления библиотек и гарантируют, что новая версия не ухудшила качество данных.

Мониторинг стабильности пайплайна

Даже идеально собранная система может деградировать со временем — накопление задержек, рост jitter, нестабильность FPS.



Метрики исполнения

Средняя latency, jitter, загрузка GPU/CPU, FPS



Визуализация

Prometheus + Grafana для сбора и отображения данных

```
while True:
    log.info(f"FPS={fps:.2f}, latency={lat:.3f}s, "
            f"GPU={gpu_load:.1f}%, CPU={cpu_load:.1f}%")
    time.sleep(1)
```

Готовое видео для глубокого обучения

1 Декодирование без потерь

Поток декодируется без потерь кадров и дропа PTS

2 Фиксированное цветовое пространство

Одинаково для всех роликов

3 Нормализованная таймбаза

Обычно 1/30 или 1/60

4 Приведённые хрома и гамма

BT.709 и линейный RGB

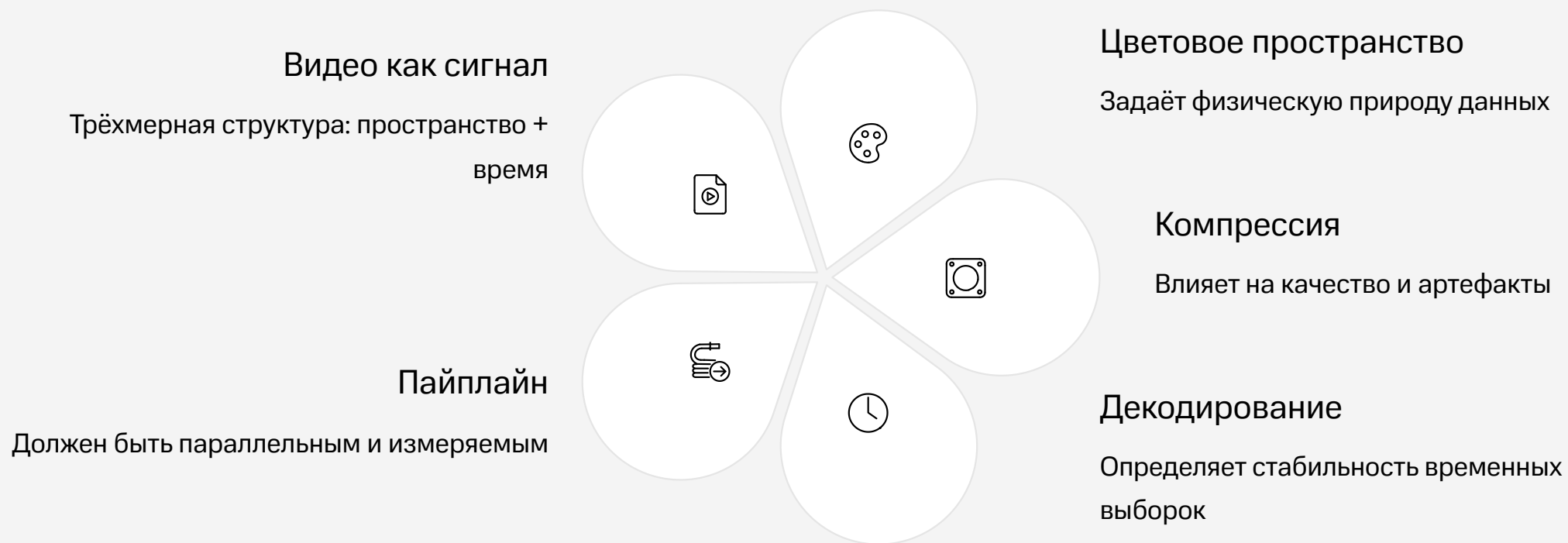
5 Стабильный FPS

Jitter < 1%

6 Тензорная форма

$T \times C \times H \times W$

Итоговые рекомендации



Инженер, понимающий эти взаимосвязи, способен проектировать устойчивые конвейеры, где каждое звено оптимизировано под реальное железо и требования latency